UNIVERSIDAD AMERICANA (UAM) • PROGRAMACIÓN ESTRUCTURADA / ESTRUCTURAS DE DATOS

JaguarFind: Especificación Técnica y Funcional

Sistema Web de Orientación en el Campus y Gestión de Objetos Perdidos mediante Grafos Ponderados, Dirigidos y No Completos

Atributo del Proyecto	Detalle Oficial
Nombre de la Aplicación	**JaguarFind**
Integrantes del Grupo	Br. Camila Javiera Navarro Centeno, Br. Francis Alejandra Solórzano Morales, Br. James Santiago Fernandez Obregon, Br. Elias Abraham Solis Guevara
Estructura de Datos	Grafo Ponderado, Dirigido y No Completo (con Algoritmo de Dijkstra)
Público Objetivo	Estudiantes, docentes, personal administrativo (Objetos Perdidos - Edificio A) y visitantes
Stack Tecnológico	Python 3 + Flask (Jinja2) + SQLite3 + HTML5 + Tailwind CSS (CDN) + HTML5 Geolocation API

1. Concepto y Propuesta Oficial Actualizada

JaguarFind es una aplicación web diseñada para facilitar la orientación dentro del campus universitario de la UAM y digitalizar de extremo a extremo la gestión de objetos perdidos y encontrados que actualmente se lleva de forma manual en un cuaderno físico en el área de Objetos Perdidos ubicada en el Edificio A.

Para garantizar una arquitectura limpia, escalable y libre de saturación de aristas, la plataforma se fundamenta en **un único grafo ponderado, dirigido y no completo** que modela el mapa físico de la universidad. Cada nodo representa un edificio o punto de interés (con sus coordenadas reales de latitud y longitud) y las aristas representan los pasillos o senderos peatonales con su distancia en metros como peso. Sobre esta estructura única operan dos módulos interconectados:

- **Módulo 1 (Orientación y Rutas en el Campus):** Permite visualizar el mapa digital de la UAM, consultar ubicaciones específicas y calcular la ruta óptima entre diferentes puntos utilizando el algoritmo de caminos mínimos de Dijkstra.
- **Módulo 2 (Gestión Inteligente de Objetos Perdidos):** Permite registrar, filtrar y administrar objetos perdidos y encontrados vinculando cada reporte directamente al nodo del grafo donde fue hallado y gestionando su custodia en el área de Objetos Perdidos ubicada en el Edificio A.

2. Diseño y Funcionamiento de la Estructura de Datos (El Grafo)

En lugar de construir un segundo grafo de relaciones entre objetos (lo cual generaría miles de conexiones redundantes si muchos objetos comparten el color negro o la misma fecha), **JaguarFind** utiliza el grafo del campus como motor espacial de toda la aplicación.

2.1. Propiedades Matemáticas del Grafo de la UAM

- **Nodos (V):** Edificios y zonas clave del campus (ej. *Edificio A - Objetos Perdidos*, *Biblioteca*, *Cafetería*, *Edificio B*, *Jaguar Center*, *Parqueo Principal*). Cada objeto Nodo en Python almacena su identificador (`idNodo`), nombre, coordenadas GPS (`latitud`, `longitud`) y una referencia a los objetos reportados en ese punto.
- **Grafo Dirigido:** Las aristas (E) poseen dirección explícita ($u \rightarrow v$). Esto permite modelar flujos reales de circulación peatonal, entradas/salidas específicas o desniveles donde ir de $A \rightarrow B$ es distinto de $B \rightarrow A$.
- **Grafo Ponderado:** Cada arista almacena un peso numérico positivo ($w > 0$) que representa la distancia física en metros (o tiempo estimado en segundos) entre dos edificios conectados.
- **Grafo No Completo:** No todos los edificios están conectados directamente entre sí por un pasillo recto; para ir de un extremo a otro del campus es obligatorio atravesar nodos intermedios.

2.2. Features del Grafo Integradas en el Sistema

1. **Feature 1 — Reconstrucción de "Ruta de Búsqueda" con Dijkstra:** Cuando un estudiante pierde una pertenencia y no sabe en qué punto exacto se le cayó, ingresa en JaguarFind los edificios que visitó durante el día (ej. *"A las 8:00 AM estuve en el Parqueo, luego fui al Edificio B y terminé en la Cafetería"*). El algoritmo de **Dijkstra** calcula el camino mínimo entre esos puntos, identificando automáticamente todos los **nodos intermedios de paso** (ej. *Plaza Central*, *Pasillo Edificio C*), y filtra en pantalla todos los objetos encontrados a lo largo de esa cadena completa de nodos.
2. **Feature 2 — Detección Automática del Nodo Actual vía GPS (Vecino Más Cercano):** Al abrir el formulario para reportar un hallazgo, el navegador captura las coordenadas GPS del estudiante y las envía a Flask. Python compara esa coordenada contra todos los nodos del grafo mediante la fórmula de distancia geográfica (Haversine/Euclidiana) y selecciona automáticamente el edificio más cercano (ej. *"Ubicación detectada en el Grafo: Biblioteca — a 12 metros"*).
3. **Feature 3 — Análisis de "Zonas Calientes" en el Reporte Semanal (Grado y Centralidad):** El módulo administrativo analiza la acumulación de reportes por nodo junto con el grado de entrada/salida de las aristas del grafo para identificar en el reporte semanal cuáles son los edificios y cruces de pasillos con mayor incidencia de pérdidas en la UAM.

2.3. ¿Por qué se Utiliza el Algoritmo de Dijkstra? (Justificación Técnica)

El algoritmo de Dijkstra fue seleccionado porque resuelve de forma exacta el problema del camino mínimo desde un nodo origen hacia un nodo destino dentro de un **grafo dirigido, ponderado y no completo con pesos no negativos ($w \geq 0$)**. En JaguarFind cumple una doble función esencial: primero, guía peatonalmente a los usuarios por la ruta más corta en metros reales dentro del campus; y segundo, permite inferir con precisión matemática por cuáles pasillos y edificios intermedios caminó un estudiante entre dos clases para rastrear pertenencias extraviadas en ese trayecto.

Algoritmo Comparado	Complejidad	Argumento Técnico para el Jurado
Dijkstra (Implementado)	$O((V + E) \log V)$	Garantiza encontrar la ruta óptima acumulando los pesos reales de las aristas (distancias en metros mayores o iguales a cero) y reconstruye la secuencia exacta de nodos intermedios mediante una cola de prioridad.
BFS / DFS	$O(V + E)$	Se descartan porque operan sobre grafos no ponderados e ignoran la distancia en metros. BFS hallaría el camino con menos edificios intermedios, aunque sea físicamente más largo.
Bellman-Ford	$O(V \times E)$	Se descarta por su menor rendimiento; está diseñado para grafos con aristas de peso negativo, las cuales no existen en distancias físicas del campus.
Floyd-Warshall	$O(V^3)$	Se descarta por su complejidad cúbica al calcular todos los pares de nodos simultáneamente, cuando una consulta de ruta o búsqueda requiere evaluar únicamente el recorrido del usuario activo.

3. Flujo Operativo de la Aplicación y Ciclo de Vida del Objeto

El sistema reemplaza el cuaderno físico del área de Objetos Perdidos ubicada en el Edificio A mediante un flujo controlado de **tres estados** que evita publicaciones anónimas falsas y agiliza la entrega.

3.1. Autenticación Institucional y Roles

- **Estudiantes:** Inician sesión con su **CIF** (exactamente 8 dígitos numéricos) y una **contraseña de 6 caracteres alfanuméricos**, vinculados a su correo institucional `@uamv.edu.ni`, nombre completo y carrera.
- **Administrador (Objetos Perdidos - Edificio A):** Cuenta con credenciales de administración que habilitan el tablero de control de tickets (*Cards*), buscador por CIF/nombre, cambio de estados y generación del reporte semanal.

3.2. Los 3 Estados de un Objeto Perdido

Estado del Ticket	Actor que lo Activa	Acción en JaguarFind y Datos Registrados
1. Reportado (En camino a Edificio A)	Estudiante que encuentra el objeto	El estudiante encuentra un objeto (ej. una botella en *Biblioteca*). Entra a la web; el sistema detecta automáticamente el nodo *Biblioteca* por GPS, toma la fecha y hora exacta con `datetime.now()` y vincula su CIF, nombre y carrera de su sesión. Solo elige Categoría, Color y descripción breve. Luego lleva el objeto físicamente al área de Objetos Perdidos en el Edificio A.
2. En Custodia (Entregado en Objetos Perdidos)	Administrador (Edificio A)	El estudiante entrega el objeto en Objetos Perdidos (Edificio A). El Admin busca el ticket en su panel de *Cards* por el CIF o nombre del estudiante, verifica el artículo físico, registra la cédula de quien entrega y cambia el estado a **"Entregado en Objetos Perdidos"**. El objeto aparece como verificado en el mapa y lista pública.
3. Entregado a su Dueño (Archivado)	Administrador (Edificio A)	El dueño localiza su pertenencia en JaguarFind (usando filtros, las alertas ! del mapa o reconstruyendo su *Ruta de Búsqueda* con Dijkstra) y acude a Objetos Perdidos en el Edificio A. El Admin abre la *Card*, registra los datos del dueño (CIF, nombre completo, carrera y cédula) y marca **"Entregado a su dueño"**. El ticket se archiva automáticamente y sale de la vista pública.

3.3. Interfaz de Consulta para Estudiantes (Vista Combinada)

Para ofrecer una experiencia intuitiva y rápida de programar en Flask + Tailwind CSS, la pantalla principal presenta una **vista dividida**:

- **Panel Izquierdo (Lista y Filtros + Ruta de Búsqueda):** Permite filtrar objetos por Categoría, Color, Fecha y Nodo de hallazgo, o bien ingresar los edificios visitados en el día para que Dijkstra filtre los objetos hallados en toda la ruta.
- **Panel Derecho (Mapa de la UAM con Alertas !):** Muestra el croquis del campus con insignias de alerta sobre cada edificio que tiene objetos reportados. Al hacer clic en un nodo del mapa, se filtra la lista izquierda para mostrar únicamente los objetos de ese edificio.

4. Catálogo Estandarizado de Categorías y Base de Datos

Para evitar errores de escritura libre, el formulario de reporte utiliza **8 categorías cerradas** acompañadas de una lista de colores principales (Negro, Blanco, Azul, Rojo, Gris/Plateado, Verde, Café, Multicolor, Otro):

1. **Electrónicos y Accesorios:** Celulares, laptops, tablets, audífonos, cargadores, USB, calculadoras científicas.
2. **Documentos e Identificaciones:** Carné universitario (CIF), cédula de identidad, licencia de conducir, tarjetas, pasaporte.
3. **Bolsos y Contenedores:** Mochilas, bolsos, carteras, billeteras, cartucheras y loncheras.
4. **Termos y Recipientes:** Botellas de agua, termos de café y recipientes de comida.
5. **Llaves y Candados:** Llaves de casa, llaves de vehículo/moto, llaveros y candados.
6. **Material Académico y Salud:** Cuadernos, libros, batas de laboratorio, estetoscopios, tubos de planos.
7. **Prendas y Accesorios Personales:** Chaquetas, suéteres, gorras, lentes, paraguas, relojes y joyería.
8. **Otros:** Artículos misceláneos con descripción obligatoria.

4.1. Esquema de Tablas en SQLite (`jaguarFind.db`) en Notación camelCase

Tabla	Columnas Principales (camelCase)	Relación con el Grafo / Sistema
`usuarios`	`cif` (PK, TEXT 8 dígitos), `nombreCompleto`, `correoInstitucional` (`@uamv.edu.ni`), `carrera`, `cedula`, `passwordHash`, `rolUsuario` (`estudiante` / `admin`)	Autenticación y autocompletado de datos al reportar o reclamar en Objetos Perdidos (Edificio A).
`objetosPerdidos`	`idTicket` (PK), `categoria`, `color`, `descripcion`, `idNodoHallazgo`, `fechaHoraHallazgo`, `estadoTicket` (1, 2 o 3), `cifEncontro` (FK), `cedulaEncontro`, `cifDueno`, `nombreDueno`, `carreraDueno`, `cedulaDueno`, `fechaEntrega`	El atributo `idNodoHallazgo` conecta cada registro de SQLite con el objeto Nodo correspondiente dentro de la clase `GrafoUam` en Python.

5. Arquitectura Tecnológica y Estructura del Proyecto (camelCase)

Toda la lógica de negocio, filtros, sesiones y algoritmos de grafos se ejecutan en el servidor con **Python y Flask** usando plantillas Jinja2 y convención de nombres en **camelCase**. El único bloque mínimo de JS en el navegador son 6 líneas para leer el GPS del dispositivo y enviarlo en el formulario de Flask.

```
jaguarFind/
├── app.py                  # Servidor Flask, rutas web, autenticación y vistas
Jinja2
├── grafoUam.py             # Clases Nodo, Arista, GrafoUam (métodos
calcularDijkstra, detectarNodoGps, obtenerZonasCalientes)
├── database.py             # Conexión SQLite3, creación de tablas y consultas CRUD
en camelCase
├── jaguarFind.db           # Base de datos relacional local (SQLite)
├── static/
│   └── img/
│       └── mapaUam.png     # Croquis base del campus de la UAM
└── templates/
    ├── base.html           # Plantilla maestra con Tailwind CSS vía Play CDN
    ├── login.html          # Inicio de sesión con CIF (8 dígitos) y contraseña (6
caracteres)
    ├── index.html          # Vista dividida: Mapa con alertas (!), Ruta Dijkstra y
Filtros
    ├── reportar.html       # Formulario rápido con detección automática de nodo
vía GPS
    └── admin.html          # Panel de Objetos Perdidos - Edificio A (Cards de
tickets y Reporte Semanal)
```

> **Fragmento HTML5 para la detección automática de ubicación en `reportar.html`:** Al cargar el formulario, el navegador rellena dos campos ocultos (`latitudActual` y `longitudActual`) con `navigator.geolocation.getCurrentPosition()`. Cuando el estudiante envía el formulario, el método `detectarNodoGps()` en Python calcula la distancia hacia cada nodo del grafo y asigna el edificio automáticamente.

6. División de Roles del Equipo y Plan de Trabajo (10 a 12 Días)

Rol / Integrante	Módulo Asignado	Entregables Específicos en el Repositorio
Rol 1: Especialista en Estructuras de Datos	Motor del Grafo (`grafoUam.py`)	Programar desde cero las clases `Nodo`, `Arista` y `GrafoUam` (dirigido, ponderado y no completo) usando notación camelCase; mapear los 12-15 edificios de la UAM con sus coordenadas y distancias en metros; implementar `calcularDijkstra()` para la Ruta de Búsqueda, `detectarNodoGps()` y `obtenerZonasCalientes()`.
Rol 2: Desarrollador Backend y Datos	Persistencia y Lógica (`database.py` y rutas en `app.py`)	Diseñar las tablas `usuarios` y `objetosPerdidos` en SQLite3 con columnas en camelCase; programar validaciones regex para CIF (8 dígitos), contraseña (6 alfanuméricos) y correo `@uamv.edu.ni`; crear las funciones `registrarObjeto()`, `filtrarPorRuta()` y `generarReporteSemanal()`.
Rol 3: Desarrollador Frontend Estudiante	Interfaz Pública y Mapa (`index.html`, `reportar.html`)	Maquetar con Tailwind CSS la vista dividida (lista filtrable + mapa de la UAM con alertas `!`); integrar el selector de edificios para reconstruir la Ruta de Búsqueda con Dijkstra y el formulario rápido de reporte con lectura automática de coordenadas GPS.
Rol 4: Desarrollador Panel Admin e Integración	Objetos Perdidos - Edificio A (`admin.html`, `login.html`)	Construir el tablero de *Cards* para el área de Objetos Perdidos en el Edificio A con buscador por CIF/nombre; programar los formularios de transición entre los 3 estados (recepción en custodia, registro de datos del dueño y archivado automático) y la vista del Reporte Semanal con Zonas Calientes.